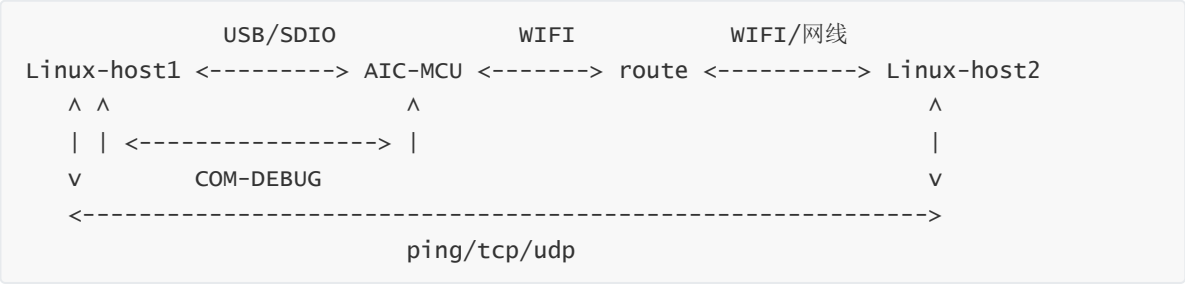


Fhostif 开发指南

一、虚拟网卡模式

1. 硬件设备连接说明



在此模式下，AIC-MCU相当于Linux-host1有线连接的以太网卡，实际在Linux-host1中访问到的设备是一个虚拟的以太网卡设备，可通过ifconfig查看

2. AIC-MCU低功耗-target描述

以 AIC8800MC 为例，在 config/aic8800mc 目录下存在多个 fhostif 相关子目录，

aic8800-sdk/config/aic8800mc/	
└─ target_wifi_fhostif	a---> wifi 功能虚拟网卡
└─ target_ble_wifi_fhostif	b---> 相较a，增加ble功能
└─ target_wifi_fhostif_ramopt	c---> 相较a，将wifi协议栈运行于flash，提供更多内存
└─ target_ble_wifi_fhostif_ramopt	d---> 相较b，将wifi协议栈运行于flash，提供更多内存

3. AIC-MCU低功耗-编译启动

AIC-MCU

AIC-MCU 代指 AIC8800x，当前 AIC8800x 芯片系列包括 AIC8800M、AIC8800MC、AIC8800M40 等

以 target_wifi_fhostif 为例，

1. 打开文件 /config/aic8800x/target_wifi_fhostif/tgt_cfg/tgt_cfg_wifi.h
2. 定义虚拟网卡模式

```
/**
 * Hostif mode selection, match with host driver
 * Current support:
 * 1) HOST_VNET_MODE
 * 2) HOST_RAWDATA_MODE
 */
#define CONFIG_HOSTIF_MODE HOST_VNET_MODE
```

3. 进入目录 `/config/aic8800x/target_wifi_fhostif`，若需要使用蓝牙功能，则进入 `/config/aic8800x/target_ble_wifi_fhostif`
4. 使用脚本 `build_fhostif_wifi_case.sh` 编译

```
# 编译usb接口版本，默认usb接口
./build_fhostif_wifi_case.sh HOSTIF=usb -j8
# 编译sdio接口版本
./build_fhostif_wifi_case.sh HOSTIF=sdio -j8
```

5. 将 `/build/host-wifi-aic8800/host_wb.bin` 烧写到 `x 8000000`
(或者 `/build/host-wifi-aic8800mc/host_wb_aic8800mc.bin` 烧写到 `x 8000000`)
6. 使用 `g 8000000` 执行MCU程序

Linux主机

Linux主机运行两部分程序，**虚拟网卡驱动 + 应用程序**

Linux主机加载驱动过程中将会注册网络设备，该网络设备的mac地址需与MCU保持一致。加载驱动时获取mac地址并直接注册网络设备，称为**非快速启动**；加载驱动时使用随机mac地址，加载完驱动才进行同步，称为**快速启动**。

a. 加载网卡驱动

1. 进入 `/wifi/LinuxDriver/aic8800_netdrv` 目录
2. 打开 **Makefile** 文件，根据主机的实际环境修改配置

```
# 主机平台默认为UBUNTU
CONFIG_PLATFORM_NANOPI_M4 ?= n
CONFIG_PLATFORM_ALLWINNER ?= n
CONFIG_PLATFORM_INGENIC_T31 ?= n
CONFIG_PLATFORM_INGENIC_T40 ?= n
CONFIG_PLATFORM_UBUNTU ?= y

# Driver mode support list
# 默认使用虚拟网卡模式
CONFIG_VNET_MODE ?= y
CONFIG_RAWDATA_MODE ?= n

# 默认关闭快速启动
CONFIG_FAST_INSMOD ?= n

# 接口默认使用USB
CONFIG_SDIO_SUPPORT = n
CONFIG_USB_SUPPORT = y
```

3. 编译加载驱动

```
make -j8
sudo insmod aic8800_netdrv.ko
ifconfig vnet0 up (T31/T40/T41需要)
```

安装驱动后, 在Linux-host1终端中输入ifconfig, 即可观察到已经注册了虚拟网卡vnet0

4. 驱动加载注意事项

sdio

加载sdio驱动之前必须先完成sdio识卡过程

1. sdio识卡 (clk、cmd、GND)

AIC8800x连接主控sdio, 主控内核加载后将会启动识卡过程。识卡过程跟驱动无关, 若sdio接口使用飞线或者卡槽连接, 可能由于接线长度不等, 导致识卡失败。调试过程中可以在clk线上接一个小电容并连接到GND, 电容大小需实际测试。若识卡成功, AIC8800x将从主控分配得到一个sdio设备地址

2. 驱动加载

同样, 若是采用飞线或者卡槽连接, sdio时钟不能设置过高, 设置过高很容易出现sdio数据包crc-err, 推荐卡槽20M左右, 飞线10M以内。此外, 降低时钟后加载sdio驱动还有可能出现主控发包成功, 收包失败的现象, 此时调整sdio相位即可

修改说明

// V2: AIC8800M/MC, V3: AIC8800M40

// V2 直接修改以下宏即可修改时钟与相位

aicwf_sdio.h 文件

```
#define SDIOWIFI_CLOCK_V2          20000000 // default 20MHz
#define SDIOWIFI_CLOCK_V3          20000000 // default 20MHz
#define SDIOWIFI_PHASE_V2          0         // 0: default, 2: 180°
```

// V3 需要打开以下代码

在 aicwf_sdiov3_func_init 接口中

```
u8 val = 0; // 去掉注释
#if 1      // 打开条件编译
if (host->ios.timing == MMC_TIMING_UHS_DDR50) {
    val = 0x21;//0x1D;//0x5;
} else {
    val = 0x01;//0x19;//0x1;
}
val |= SDIOCLK_FREE_RUNNING_BIT;
sdio_f0_writeb(sdiodev->func, val, 0xF0, &ret);
if (ret) {
    sdio_err("set iopad ctrl fail %d\n", ret);
    sdio_release_host(sdiodev->func);
    return ret;
}
sdio_f0_writeb(sdiodev->func, 0x0, 0xF8, &ret);
if (ret) {
    sdio_err("set iopad delay2 fail %d\n", ret);
    sdio_release_host(sdiodev->func);
    return ret;
}
# 修改寄存器0xF1值, 可设置0x10/0x20/0x40/0x80/0xF0五个相位
sdio_f0_writeb(sdiodev->func, 0x20, 0xF1, &ret);
if (ret) {
    sdio_err("set iopad delay1 fail %d\n", ret);
```

```

        sdio_release_host(sdiodev->func);
        return ret;
    }
    msleep(1);
    #if 1//SDIO CLOCK SETTING
    if (host->ios.timing != MMC_TIMING_UHS_DDR50) {
        host->ios.clock = SDIOWIFI_CLOCK_V3; # aicwf_sdio.h中修改
        host->ops->set_ios(host, &host->ios);
    }
    #endif
    #endif

```

b. 启动虚拟网卡

非快速启动

非快速启动，即正常模式，此模式下加载驱动，先注册完`sdio`设备后，主控需要从`mcu`获取联网状态以及`mac`信息，之后才能注册网卡设备

1. 进入 `/wifi/LinuxDriver/app/custom_msg` 目录直接编译
(若嵌入式平台，则需使用相应的交叉编译工具进行编译)
2. 连接路由器 AP

```

./custom_msg vnet0 1 ssid password
例如，
./custom_msg vnet0 1 XIAOMI_HHH 1234567

```

3. 通过串口观察 **MCU-DEBUG** 信息，接受连接指令后，将连接上路由，并分配得到 **mcu-ip + mcu-gw** 通过MCU的串口即可观察到
以下设置均跟 **mcu-ip + mcu-gw** 相关
4. 设置虚拟网卡 ip 与 gw

```

sudo ifconfig vnet0 <mcu-ip> (ie: 192.168.3.36)
sudo route add default gw <mcu-gw> (ie: 192.168.3.1)

```

5. 设置网络 DNS

```

# 打开resolv.conf文件
sudo chmod 777 /etc/resolv.conf
vim /etc/resolv.conf

# 添加如下内容，注意，Linux-host1系统重启之后该文件会复原
namespace mcu-gw (ie: 192.168.3.1)

```

6. 测试网络联通

```
# 实际中，依据不同平台，若通过步骤4即可ping通网络，则步骤5无需操作

# 路由
ping 192.168.12.1

# 局域网
ping 192.168.12.10

# 外网
ping 202.108.22.5
```

快速启动

再次说明：指系统加载驱动过程中，不获取MCU返回的mac地址，使用软件随机mac地址注册vnet_dev，驱动加载完成之后，再利用应用程序获取MCU的mac地址，并同步到vnet_dev网络设备

1. 修改 Makefile 设置

```
# 默认关闭快速启动
CONFIG_FAST_INSMOD ?= y
```

2. 加载完驱动后，同步MCU的mac地址

```
# 获取MCU的mac地址，通过dmesg查看信息
custom_msg vnet0 5

# 同步vnet_dev的mac地址
sudo custom_msg vnet0 ndev mac_address
ie: sudo custom_msg vnet0 ndev 00:11:22:33:44:55

# 或者，使用ifconfig命令修改mac地址
sudo ifconfig vnet0 down
sudo ifconfig vnet0 hw ether 00:11:22:33:44:55
sudo ifconfig vnet0 up
```

3. 其余操作同 非快速启动

c. 应用程序

在 /wifi/LinuxDriver/app/ 目录下，有 custom_msg、fasync_demo 两个应用程序

custom_msg

用于往主控驱动中下发指令，并通过驱动中的SDIO/USB接口发送到 AIC-MCU，custom_msg 与 AIC-MCU 之间的交互指令如下，

```
# 编译完成应用程序之后，输入./custom_msg，即可看到指令说明
"usage: custom_msg vnet0 [mode] <arg1> <arg2> <arg3>"
"-----"
">>>Interact with MCU:"
"custom_msg vnet0 1 ssid password          - connect ap"
"custom_msg vnet0 2                        - disconnect ap"
"custom_msg vnet0 3                        - close ble before sleep if used\r\n"
"custom_msg vnet0 4                        - enter sleep"
"custom_msg vnet0 5                        - exit sleep"
"custom_msg vnet0 6                        - get mcu mac"
"custom_msg vnet0 7                        - get wlan status"
```

```

"custom_msg vnet0 8 ssid password band      - start ap"
"                                           -- band = <2.4G/5G>"
"custom_msg vnet0 9                          - change ap mode\r\n"
"custom_msg vnet0 10                         - stop ap"
"custom_msg vnet0 11                         - scan wifi"
"custom_msg vnet0 12 /your-path/update.bin - host ctrl OTA"
"-----"
">>>Interact with kernel:"
"sudo custom_msg vnet0 ndev mac_address      - set vnet_dev mac"

```

注意要点

1. **消息阻塞** 驱动可以针对 **custom_msg** 消息 id 进行阻塞与否的配置

```

// handle_custom_msg 函数
case APP_CMD_CONNECT:
    .../
    // 消息阻塞 AICWF_CMD_WAITCFM
    cust_app_cmd.cmd_cfm.waitcfm = AICWF_CMD_WAITCFM;
    // 等待回应 CUST_CMD_CONNECT_IND
    cust_app_cmd.cmd_cfm.cfm_id = CUST_CMD_CONNECT_IND;
    ret = rwnx_tx_msg((u8 *)&cust_app_cmd.connect_req,
sizeof(cust_app_cmd.connect_req), &cust_app_cmd.cmd_cfm, command);
    break;

case APP_CMD_SCAN_WIFI:
    printk("APP_CMD_SCAN_WIFI\n");
    cust_app_cmd.common_req.cmd_id = CUST_CMD_SCAN_WIFI_REQ;
    // 不阻塞
    cust_app_cmd.cmd_cfm.waitcfm = AICWF_CMD_NOWAITCFM;
    // 不等待回应
    cust_app_cmd.cmd_cfm.cfm_id = 0;
    ret = rwnx_tx_msg((u8 *)&cust_app_cmd.common_req,
sizeof(cust_app_cmd.common_req), &cust_app_cmd.cmd_cfm, command);
    break;

```

2. **超时时间** 驱动中对 **custom_msg** 消息阻塞时间做了设置，用户可自行更改

```

// rwnx_main.h 注意，若需返回联网结果，超时时间应设置大于联网所需时间
#define AICWF_CMDCFM_TIMEOUT      2000

```

3. **结果回显** 对于阻塞的 **custom_msg** 消息，返回时将带有反馈消息，若不需要直接注释即可

```

// custom_msg.c
// If don't need return-result, it's OK to comment out this printf.
printf(APP_NAME "%s\n", (char *)&priv_cmd.buf[0]);

```

fasync_demo

利用字符设备的异步消息机制，实现驱动主动向应用层发送消息的功能。**fasync** 功能默认关闭，启动该功能，需要修改驱动 Makefile，之后在驱动加载过程中将会注册用于消息发送的字符设备，加载完驱动，需将 **fasync_user** 运行在后台实时接收消息

```
# Msg Callback setting 默认为n
CONFIG_APP_FASYNC ?= y
```

注意要点

1. **后台运行 fasync_user** 应该在 **aic8800_netdrv.ko** 驱动加载后，**custom_msg** 下发指令之前启动，并且一直保持在后台运行
2. **消息回写** 在驱动连接多次发送消息的情况下，应用层来不及处理消息，可能导致出现消息遗漏的问题。为此，字符设备消息的发送过程，要求应用层收到消息之后，进行状态位的设置，之后驱动才能够进行下一次消息的发送。用户在移植时应注意。

```
static void signal_handler(int signum)
{
    int ret = 0;
    char data_buf[sizeof(struct rwnx_fasync_info)];
    struct rwnx_fasync_info *fsy_info = (struct rwnx_fasync_info *)data_buf;
    ret = read(fd, data_buf, sizeof(struct rwnx_fasync_info));
    if(ret < 0) {
        printf(APP_NAME "Read kernel-data fail\n");
    } else {
        printf(APP_NAME "%s\n", fsy_info->mem);
        // 设置状态 0，表示已经收到了消息
        fsy_info->mem_status = 0;
        ret = write(fd, &fsy_info->mem_status, sizeof(fsy_info->mem_status));
        // 回写设备 buff
        if (ret < 0)
            printf(APP_NAME "Write kernel-data fail\n");
        // 解析内核消息
        analy_signal_msg(fsy_info->mem);
    }
}
```

若没有启动 **fasync_user** 程序，或者该程序中没有设置 **mem_status**，则驱动往应用层发送消息将会等待超时，才能发送下一条消息。

```
# rwnx_main.h 可调整超时时间
#define FASYNC_APP_TIMEOUT 100
```

3. **自动配网 fasync_user** 解析到联网信息时，将自动提取 **ip** 和 **gw**，通过 **system** 接口自动配置主控网络状态，用户可针对具体需求解析消息并作相应配置

```
void analy_signal_msg(char *mem)
{
    char buff[64];
    struct wlan_settings wlan;
    // auto wlan-settings
    char *ptr = strstr(mem, "3003");
    if (ptr) {
        // set vnet0 ip
        ptr = strstr(mem, "ip");
        memcpy(buff, ptr+4, WLAN_SET_LEN);
        wlan.ip = str2uint(buff);
        sprintf(buff, "ifconfig vnet0 %d.%d.%d.%d",
```

```

        (unsigned int)((wlan.ip >> 0)&0xFF), (unsigned int)
((wlan.ip >> 8)&0xFF),
        (unsigned int)((wlan.ip >> 16)&0xFF), (unsigned int)
((wlan.ip >> 24)&0xFF));
        system(buff);

        // set vnet0 gw
        memcpy(buff, ptr+18, WLAN_SET_LEN);
        wlan.gw = str2uint(buff);
        sprintf(buff, "route add default gw %d.%d.%d.%d",
                (unsigned int)((wlan.gw >> 0)&0xFF), (unsigned int)
((wlan.gw >> 8)&0xFF),
                (unsigned int)((wlan.gw >> 16)&0xFF), (unsigned int)
((wlan.gw >> 24)&0xFF));
        system(buff);
        system("ifconfig");
    }
}

```

4. **设备号冲突** 若因为开启了CONFIG_APP_FASYNC，导致加载驱动失败，可以先确认下是否因为字符设备号冲突了，按照如下修改即可

```

# 相关指令
ls -l /dev          查看设备文件号
cat /proc/devices   查看设备号
ls /sys/class        查看字符设备class

```

4. AIC-MCU低功耗-验证流程

1. Linux-Host1 在没有执行任务时自动关闭，并在需要执行任务时进行唤醒，恢复工作状态
2. 在AIC-MCU端进入休眠之后，可通过对EVB板的电流进行实时监控，测量应用中AIC-MCU休眠功耗
3. 用于测量功耗的EVB板必须**断开外部器件**，使用vbat供电，测量核心供电以免功耗数据偏高

验证流程可参考如下

以下仅为验证demo，用户需要根据产品实际应用场景进行修改，具体请结合SDK中，**aic8800_netdrv**驱动与**fhostif_cmd.c**应用代码进行分析

1. 上电运行 AIC-MCU 与 Linux-Host1，其中 AIC-MCU 可切换成正常模式，上电将自动执行
2. Linux-Host1 加载驱动

```

# 主控操作
sudo insmod aic8800_netdrv.ko

```

3. AIC-MCU 配网，当前支持三种模式，
 - 1) 通过 **AIC-MCU 串口交互指令**进行配网
 - 2) 通过 Linux 端**应用层交互程序** wifi/LinuxDriver/app/custom_msg 向驱动发送指令，控制 MCU 配网


```
# 串口指令
connect 0 ssid password
# 主控指令
./custom_msg vnet0 1 ssid password
```

3) 通过蓝牙配网 (编译带有蓝牙的target)

```
# fhostif_example.c do_blewifi_config 蓝牙配网例程
1. 通过串口启动, ble_wifi_cfg
2. 修改例程, 在适当时机启动配网
```

4. Linux-Host1 获取 MCU 配网信息并配置网络, 执行相关任务

5. Linux-Host1 空闲, 通过应用程序下发指令告知MCU进入休眠, 同时主控驱动将停止接收上层应用数据包

```
# 关闭蓝牙, 蓝牙功能会影响wifi休眠
如果编译了蓝牙功能, 进入休眠之前必须先关闭蓝牙
./custom_msg vnet0 3
# 主控指令
./custom_msg vnet0 4
```

6. Linux-Host1 卸载驱动, 并掉电关闭

```
# 主控操作
sudo rmmod aic8800_netdrv
```

7. AIC-MCU 收到进入休眠指令, 将设置低功耗唤醒源

```
# MCU操作
# 体现于 custom_msg_enter_sleep_handler
sleep_level_set(...); // 设置休眠等级
user_sleep_wakesrc_set(...); // 设置唤醒源
user_sleep_allow(1); // 允许休眠
```

8. AIC-MCU 将执行 **host_if_poweroff** 函数, 关闭 SDIO/USB 接口

host_if_poweroff不能在custom_msg_enter_sleep_handler中调用, 这将可能导致buff出问题
例程中, host_if_poweroff在timer_handler中被调用

```
# 串口指令
# 注意: 测试过程中可在MCU串口上关闭接口, 但这种方式无法停止主控驱动继续往MCU接口
(SDIO/USB)发送数据包
hpof
```

9. AIC-MCU 进入低功耗, 并进行其他相关保活操作

```
# 低功耗状态下应用开发说明
1. MCU休眠一段时间后, 定时唤醒执行相关任务
2. 若有用户应用需要执行, MCU将在执行完相关任务才进入休眠
```

10. AIC-MCU 被相关事件唤醒, 退出低功耗, 并通过指定 IO 给主控上电

```
# MCU操作
user_sleep_allow(0);           // 不允许休眠
sleep_level_set(PM_LEVEL_ACTIVE); // 设置活跃等级
# IO上电，demo中未体现，客户自行添加
```

11. AIC-MCU 恢复与主控接口，执行 **host_if_repower** 函数，开启 SDIO/USB 接口

```
# 串口指令
hpon
```

12. Linux-Host1 上电启动，重新加载驱动

```
# 主控操作
sudo insmod aic8800_netdrv
```

13. Linux-Host1 获取 AIC-MCU 配网信息并配置网络，重新执行任务

5. AIC-MCU低功耗-状态说明

依据AIC-MCU是否连接主控，msg是否连通，data是否连通，定义AIC-MCU与主控接口的四种状态

```
# 定义 AIC-MCU 与 Linux-Host1 接口的状态
typedef enum hostif_status
{
    HOSTIF_ST_INIT      = 0, // host power off -> null msg, null data
    HOSTIF_ST_IDLE      = 1, // host power on  -> tx/rx msg, null data
    HOSTIF_ST_AWAKE     = 2, // host power on  -> tx/rx msg, tx/rx data
    HOSTIF_ST_DEEPSLEEP = 3, // host power off -> null msg, null data
} hostif_status_e;
```

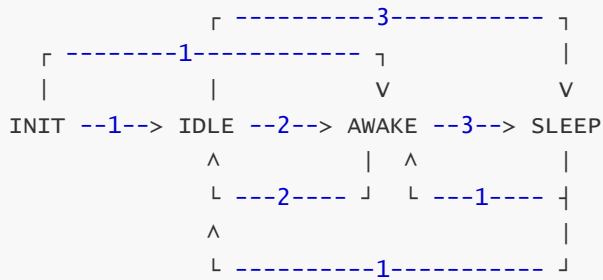
状态	描述
HOSTIF_ST_INIT	MCU-AIC刚上电，未连接主控，data与msg均不通
HOSTIF_ST_IDLE	连接主控，未配网，只有msg通
	set_hostif_wlan_status(HOSTIF_ST_IDLE);
HOSTIF_ST_AWAKE	连接主控，已配网，data与msg均通
	set_hostif_wlan_status(HOSTIF_ST_AWAKE);
HOSTIF_ST_DEEPSLEEP	未连接主控，已配网，data与msg均不通
	set_hostif_wlan_status(HOSTIF_ST_DEEPSLEEP);

HOSTIF 状态通过 **set_hostif_wlan_status** 接口进行设置，例程已经在驱动加载状态以及网络联通状态发生变化时，对 **HOSTIF** 进行了相应的设置

STA模式

```
# INIT -> HOSTIF_ST_INIT
# IDLE -> HOSTIF_ST_IDLE
# AWAKE -> HOSTIF_ST_AWAKE
# SLEEP -> HOSTIF_ST_DEEPSLEEP
```

MCU-AIC状态转移1：不启动编译softAP功能



转移条件说明

1. custom_msg_get_wlan_status_handler

主控加载驱动时必须调用该handler，以获取AIC-MCU配网状态

2. custom_msg_connect_status_ind_handler

以及custom_msg_disconnect_status_ind_handler

在连接主控之后，AIC-MCU连网与断网将会调用这两个handler

3. custom_msg_enter_sleep_handler

主控下发休眠指令后，调用该handler进入休眠

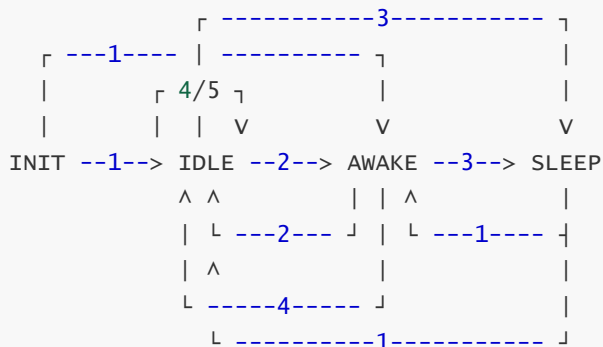
AP模式

fhostif 提供两种模式的 AP

模式	描述	HOSTIF_ST
AIC_AP_MODE_CONFIG	softAP 数据流只在 MCU 本地处理，用于AP配网	HOSTIF_ST_IDLE
AIC_AP_MODE_DIRECT	softAP 数据流与主控联通，可用于数据收发	HOSTIF_ST_AWAKE

```
# INIT -> HOSTIF_ST_INIT
# IDLE -> HOSTIF_ST_IDLE
# AWAKE -> HOSTIF_ST_AWAKE
# SLEEP -> HOSTIF_ST_DEEPSLEEP
```

MCU-AIC状态转移2：启动编译softAP功能-AIC_AP_MODE_CONFIG



转移条件说明

1~3 同上STA模式

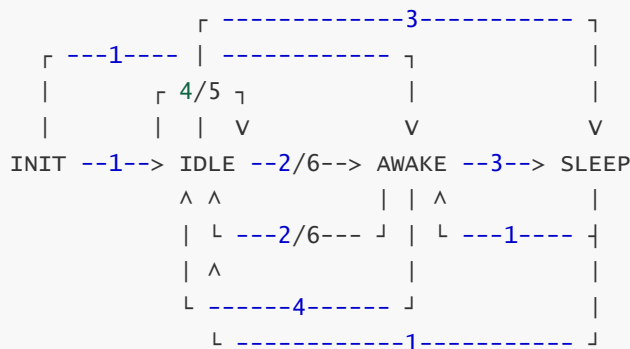
4. custom_msg_start_ap_handler

主控下发启动AP指令，如果此刻AIC-MCU正连接路由器，将会先断开路由器，返回IDLE状态

5. custom_msg_stop_ap_handler

主控下发停止AP指令

MCU-AIC状态转移3：启动编译softAP功能-AIC_AP_MODE_DIRECT



转移条件说明

1~3 同上STA模式

4. custom_msg_start_ap_handler

主控下发启动AP指令，如果此刻AIC-MCU正连接路由器，将会先断开路由器，返回IDLE状态

5. custom_msg_stop_ap_handler

主控下发停止AP指令

6. custom_msg_change_ap_mode_handler

主控下发切换AP模式指令

数据解析基本过程

结合上述AIC-MCU低功耗-验证流程，不考虑被过滤数据包，一般数据包解析过程



以上过程持续循环，直到系统关闭，详细可见demo中对 **set_hostif_wlan_status** 接口调用设置

6. AIC-MCU低功耗-数据过滤

fhostif 提供两种数据包过滤模式，**VNET_FILTER_DIRECT** 和 **VNET_FILTER_SHARED**，如上AIC-MCU低功耗-状态说明，**HOSTIF_ST_STATUS** 设置为 **HOSTIF_ST_AWAKE**，AIC-MCU才会将数据包透传给主控解析，设置过滤规则可以将满足条件的数据包保留在 AIC-MCU 中进行处理

过滤结果

在 *fhostif* 虚拟网卡模式下，AIC8800x 每次从 wifi 固件接收到数据包都需要进行过滤规则匹配，以确定当前数据包的处理，过滤结果有如下枚举

```

/** VNET filter result */
enum VNET_FILTER_RESULT
{
    VNET_TRANSFER_MCU_HOST,      // 数据包发给主控处理，同时MCU复制一份数据
    VNET_TRANSFER_MCU_ONLY,      // 数据包只给MCU处理
    VNET_TRANSFER_HOST_ONLY,     // 数据包只发给主控处理
    VNET_TRANSFER_RESEND_ONLY,   // 数据包再次转发到空中，AP模式下适用
};

```

设置过滤模式

```

// 在每个 fhostif-target 目录下均有文件 target_xxx_fhostif/tgt_cfg/tgt_cfg_wifi.h
/**
 * Hostif rx pkt filter mode
 * Current support:
 * 1) VNET_FILTER_DIRECT
 * 2) VNET_FILTER_SHARED
 */
#define CONFIG_HOSTIF_FILTER_MODE VNET_FILTER_DIRECT

```

VNET_FILTER_DIRECT

这种模式只能根据数据包中的 dst_port 和 protocol 进行判断，区分数据包发往主控，还是本地处理，也即是，过滤规则不包含**VNET_TRANSFER_MCU_HOST**

```

// set ip pkt filter
filter.used = 1;
filter.protocol = 17; // UDP
filter.dst_port = 68; // DHCP client
ret = set_hostif_user_filter(&filter);
if (ret) {
    dbg("failed to set ip pkt filter\n");
}

```

VNET_FILTER_SHARED

这种模式提供了多种的过滤规则，在设置过滤器时，通过**filter_mask**设置规则指定

过滤规则	mask
src_ipaddr	PACKET_FILTER_MASK_SRC_IP
src_ipaddr && src_port	PACKET_FILTER_MASK_SRC_IP PACKET_FILTER_MASK_SRC_PORT
protocol && src_port	PACKET_FILTER_MASK_PROTOCOL PACKET_FILTER_MASK_SRC_PORT
protocol && dst_port	PACKET_FILTER_MASK_PROTOCOL PACKET_FILTER_MASK_DST_PORT
src_ipaddr, protocol && src_port	PACKET_FILTER_MASK_SRC_IP PACKET_FILTER_MASK_PROTOCOL PACKET_FILTER_MASK_SRC_PORT

```

// set ip pkt filter
filter.used = 1;
filter.protocol = 17; // UDP
filter.dst_port = 68; // DHCP client
filter.filter_mask = PACKET_FILTER_MASK_PROTOCOL | PACKET_FILTER_MASK_DST_PORT;
ret = set_hostif_user_filter(&filter);
if (ret) {
    dbg("failed to set ip pkt filter\n");
}

```

此外，这种模式针对 **ping包** 做共享处理，当 AIC-MCU 联网之后，无论主控还是MCU都可以 ping 通外部网络，而外部网络发起 ping 请求时，将由 AIC-MCU 进行回应

代码实现

AIC8800x 接收 wifi 固件数据包接口位于 rwnx_rx.c 文件中，为方便客户使用，此文件已源码开放

```

uint8_t rwnx_rxdataind(void *pthis, void *hostid)
{
    // 接收数据包解析
    ...
    // VNET_FILTER_DIRECT 过滤模式
    if (hostif_filter_type_get() == VNET_FILTER_DIRECT) {
        // 判断HOSTIF状态，并进行rwnx_rxbuf_filtered过滤
        if ((hostif_mode_get() == HOST_VNET_MODE) &&
            (get_hostif_wlan_status() == HOSTIF_ST_AWAKE) &&
            (!rwnx_rxbuf_filtered(buf))) {
            // 与主控数据通路联通
            ...
            send_pkg_to_host(info);
            ...
        }
        // VNET_FILTER_SHARED 过滤模式
    } else if (hostif_filter_type_get() == VNET_FILTER_SHARED) {
        // 判断HOSTIF状态，
        if ((hostif_mode_get() == HOST_VNET_MODE) && (get_hostif_wlan_status()
== HOSTIF_ST_AWAKE)) {
            // 与主控数据通路联通
            ...
            // 进行rwnx_rxbuf_shared_filtered过滤
            uint8_t filter_result = rwnx_rxbuf_shared_filtered(buf, &da,
&machdr_len);
            // 根据过滤结果对数据包进行处理
            if (filter_result == VNET_TRANSFER_MCU_HOST) {
            } else if (filter_result == VNET_TRANSFER_MCU_ONLY) {
                ...
            } else if (filter_result == VNET_TRANSFER_HOST_ONLY) {
                ...
            } else if (filter_result == VNET_TRANSFER_RESEND_ONLY) {
                ...
            } else {
                ...
            }
        } else {
            // 与主控数据通路不通，数据包由MCU处理
            #if !(AICWF_RX_REORDER)
            fhost_rx_buf_forward(buf);

```

```

        #else
        rwnx_rxdataind_aicwf(buf);
        #endif /* AICWF_RX_REORDER */
    }
}

// 数据包由MCU进行处理
fhost_rx_buf_forward(buf);
...
}

```

rwnx_rxbuf_filtered 与 **rwnx_rxbuf_shared_filtered** 接口均位于 **rwnx_rx.c** 文件中，设置过滤规则的接口位于 **fhostif_cmd.c** 中，

```

int hostif_application_init(void)
{
    int ret = 0;
    ip_packet_filter_t filter;
    init_hostif_user_filter();

    if (hostif_filter_type_get() == VNET_FILTER_DIRECT) {
        // set ip pkt filter
        #if !(HOST_CNTRL_DHCP)
        filter.used = 1;
        filter.protocol = 17; // UDP
        filter.dst_port = 68; // DHCP client
        ret = set_hostif_user_filter(&filter);
        if (ret) {
            dbg("failed to set ip pkt filter\n");
        }
        #endif
        ...
    } else if (hostif_filter_type_get() == VNET_FILTER_SHARED) {
        // set ip pkt filter
        filter.used = 1;
        filter.protocol = 17; // UDP
        filter.dst_port = 68; // DHCP client
        filter.filter_mask = PACKET_FILTER_MASK_PROTOCOL |
PACKET_FILTER_MASK_DST_PORT;
        ret = set_hostif_user_filter(&filter);
        if (ret) {
            dbg("failed to set ip pkt filter\n");
        }
        ...
    }
    ...
}

// 最新代码已经将filter设置例子，统一添加到hostif_filter_setting()

```

通过 **set_hostif_user_filter** 设置的过滤规则均可以在 **rwnx_rxbuf_filtered** 与 **rwnx_rxbuf_shared_filtered** 接口中找到对应的解析代码，客户也可以进一步添加自己需要的过滤规则，需要注意的是，由于 AIC8800x 接收到 wifi 固件的每个数据包都需要对所有的过滤规则进行匹配，过滤规则不宜设置过多，否则可能会影响到 rx 的接收速率；已添加的 filter 规则可以通过 **clear_hostif_user_filter** 进行清除，也可以通过 **show_hostif_user_filter** 输出已经添加的 filter

7. AIC-MCU低功耗-接口说明

上述从几个方面，描述了虚拟网卡的工作过程，本小节将汇总描述下关键节点以及接口

1. `host_driver_t driver_st;`

变量`driver_st`定义了主控驱动的加载状态

a. 在上述3小节中，描述过虚拟网卡驱动“非快速启动”模式启动时，加载完`sdio`设备后，需要从AIC8800x获取到`mac`以及联网信息，以及才能

注册网络设备，因此，当AIC8800x收到主控下发的获取联网信息`msg`时，我们可以认为驱动已经加载完毕，硬件总线（`sdio/usb`）数据包

收发已经联通；在`custom_msg_get_wlan_status_handler`中，`driver_st` 被设置为 `HOST_DRIVER_INSMOD`

b. 收到主控下发进入休眠模式时，在`custom_msg_enter_sleep_handler`中，`driver_st` 被设置为 `HOST_DRIVER_RMMOD`

2. `void set_hostif_wlan_status(hostif_status_e st);`

这个接口设置`HOSTIF`状态，在驱动加载状态，以及网络状态发生变化时，均会进行设置，在`rwnx_rx.c`的 `rwnx_rxdaind`中，

也即AIC8800x接收`wifi`数据包接口中，我们通过`HOSTIF`状态来判断，此时是否应该将数据包发给主控

a. `HOSTIF` 默认状态为 `HOSTIF_ST_IDLE`

b. `sta`模式成功连接`ap`时，设置为 `HOSTIF_ST_AWAKE`

c. 启动`ap`模式并设置成`AIC_AP_MODE_DIRECT`模式时，设置为 `HOSTIF_ST_AWAKE`

d. `sta`模式断开`ap`时，设置为 `HOSTIF_ST_IDLE`

e. `sta`模式进入休眠时，设置为 `HOSTIF_ST_DEEPSLEEP`

f. `sta`模式退出休眠时，根据此时是否联网，设置为 `HOSTIF_ST_IDLE` 或者 `HOSTIF_ST_AWAKE`

3. `host_if_repower/host_if_poweroff`

这两个接口用于初始化硬件接口，与关闭硬件接口，硬件接口目前指`sdio/usb`外设

a. AIC8800x上电运行时自动调用`host_if_repower`接口初始化外设接口，客户不需要额外添加调用

b. 主控下发进入休眠指令后，AIC8800x需要调用`host_if_poweroff`关闭外设接口

c. AIC8800x被唤醒，退出休眠模式之后，需要调用`host_if_repower`打开外设接口，之后主控驱动才能重新加载

4. `void process_wlan_status_change(struct hostif_msg *msg)`

`sta`模式下，AIC8800x连接上`ap`与断开`ap`的回调均在这里进行处理，接口中通过

`custom_msg_connect_status_ind_handler` 向主控上报成功连接AP

`custom_msg_disconnect_status_ind_handler` 向主控上报断开AP

5. `void hostif_sdio_init_timeout_callback(void)`

使用sdio外设接口模式下，由于sdio驱动加载在某些因素影响下，可能出现加载失败的情况，为此SDK中提供了一个加载失败的回调接口

a. 使能该功能，需要在hostfi_user.c文件中配置超时参数，默认配置0，则不会触发超时

```
const uint32_t sdio_ioen_timeout_us = 0; // time unit: us, value 0 for loop forever
```

b. 此处判断sdio驱动加载失败的方式，是通过超时进行判断，当主控开始进行sdio扫卡时，AIC8800x将开始计时

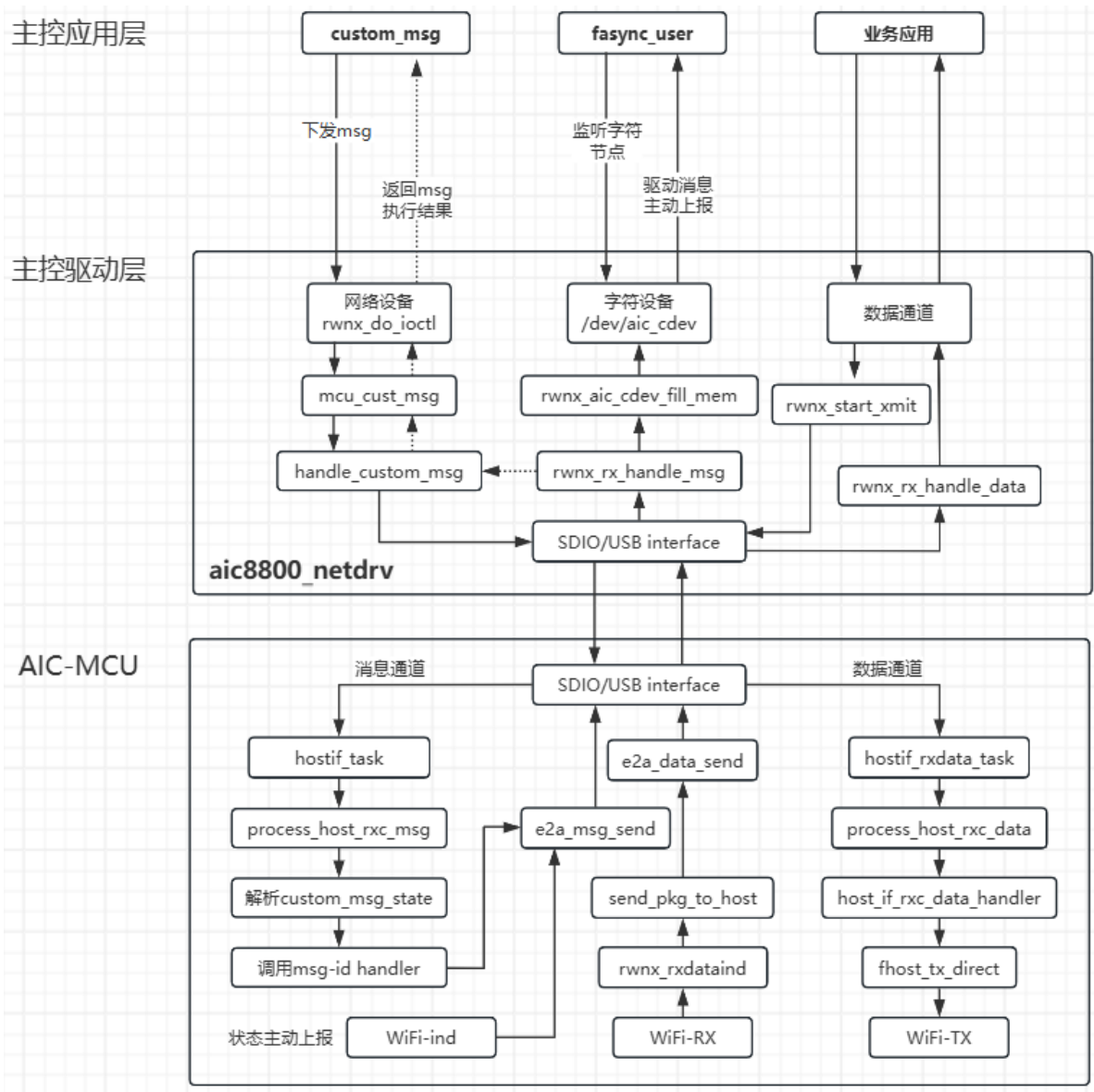
c. hostif_sdio_init_timeout_callback回调接口里边默认使用了reboot，客户可以根据产品的实际需求进行修改

8. AIC-MCU低功耗-基础架构

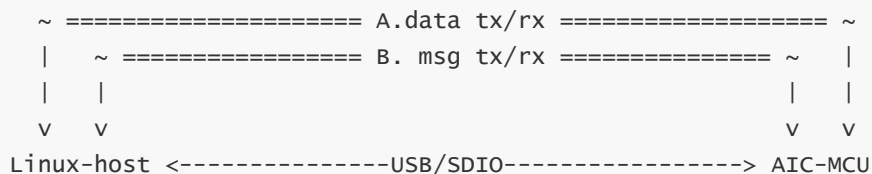
虚拟网卡结构如下图所示，主控包含两个应用 custom_msg、fasync_user，主控消息通过 custom_msg 下发消息请求（REQ），netdrv 接收后通过底层 SDIO/USB 接口传输到 AIC8800-MCU，AIC8800-MCU 一般会返回确认消息（CFM），主控驱动接收确认消息之后，返回到 custom_msg，同时也可通过 fasync_user 上报到应用层；而如果 AIC8800-MCU 主动上报的消息，netdrv 驱动只能上报到 fasync_user

基于 sdio/usb 外设接口，主控和 AIC8800-MCU 之间包含消息通道和数据通道，消息通道用于实现主控和AIC8800-MCU 之间实现应用逻辑控制，消息通道流对应主控 custom_msg、fasync_user、网络设备回调接口 ioctl 以及 AIC8800-MCU 中的 hostif_task；数据通道用于实现主控业务数据流，数据通道流对应主控 rwnx_start_xmit、rwnx_rx_handle_data 以及 AIC8800-MCU 中的 hostif_rxdarta_task

消息通道和数据通道的底层实现已经屏蔽了硬件差异，用户直接调用上层应用收发接口即可



9.AIC-MCU低功耗-收发通路



在虚拟模式网卡下，AIC-MCU 通过 USB/SDIO 接口与主控进行通信，基于该接口，SDK 实现了两个收发通路

通路A 用于收发应用数据包，AIC-MCU 接收主控的数据包为802.3格式，而 AIC-MCU 发往主控的数据包为802.11格式，之后主控在虚拟网卡驱动中将接收到802.11数据包转化成802.3格式，再上传给应用层

通路B 用于消息包收发，实现主控与 AIC-MCU 之间的应用逻辑控制和消息上报

收发通路buffer-pool

为实现数据包的快速转发，数据通路使用的是独立于 heap 的静态 buffer，该静态 buffer **hostif-pool** 的 buffer-level 定义详情如下，

level	size	describe	number	describe
1	64	可调整	8	可调整
2	128	可调整	4	可调整
3	1624	不可调整	CONFIG_HOSTIF_BUF_NUM_TOTAL	定义于target目录下的tgt_cfg_wifi.h文件

整个 **hostif-pool** 使用的内存块在 **map** 文件中定义为 **hostif_buf_pool**，虚拟网卡 target 例程所使用的map文件为 **map_cm4_fhostif_wifi.txt** 或者 **map_ble_cm4_fhostif_wifi.txt**

注意 以上三个 **buffer-level** 的 **size*number+对齐字节 <= hostif_buf_pool的LENGTH**，否则编译报错

```
MEMORY
{
    IROM (rx)          : ORIGIN = CODE_START_ADDR, LENGTH = 1224K
    #if !defined(CFG_WIFI_HIB)
    #if defined(CFG_FLASH_FW)
    DRAM (rwx)          : ORIGIN = 0x00148C00, LENGTH = 541K - 512
    IRAM (rwx)          : ORIGIN = 0x00138400, LENGTH = 0x800
    BSS2 (rwx)          : ORIGIN = 0x001CFE00, LENGTH = 512
    ipc_shared_memory (!rx) : ORIGIN = 0x00138180, LENGTH = 0x280
    ipc_pkt_memory    (!rx) : ORIGIN = 0x00138C00, LENGTH = 32K
    host_rxbuf_memory (!rx) : ORIGIN = 0x00140C00, LENGTH = 32K
    hostif_buf_pool    (!rx) : ORIGIN = 0x001D0000, LENGTH = 32K
    fmacfw_rom         (!rx) : ORIGIN = FMACFW_START_ADDR, LENGTH = 320K
    wifi_runtime_memory(!rx) : ORIGIN = 0x00100000, LENGTH = 0x38000
    wifi_setting_memory(!rx) : ORIGIN = 0x00138000, LENGTH = 0x180
    #elif defined(CFG_DPD)
    DRAM (rwx)          : ORIGIN = 0x00191400, LENGTH = 251K
    IRAM (rwx)          : ORIGIN = 0x00180400, LENGTH = 0x800
    BSS2 (rwx)          : ORIGIN = 0x00180C00, LENGTH = 512
    ipc_shared_memory (!rx) : ORIGIN = 0x00181000, LENGTH = 0x280
    ipc_pkt_memory    (!rx) : ORIGIN = 0x00181400, LENGTH = 32K
    host_rxbuf_memory (!rx) : ORIGIN = 0x00189400, LENGTH = 32K
    hostif_buf_pool    (!rx) : ORIGIN = 0x001D0000, LENGTH = 32K
    fmacfw_rom         (!rx) : ORIGIN = FMACFW_START_ADDR, LENGTH = 304K
    wifi_setting_memory(!rx) : ORIGIN = 0x00180E00, LENGTH = 0x200
    #else
```

如上图蓝色标记，若 **DRAM** 中 **ORIGIN + LENGTH** 等于 **hostif_buf_pool** 的 **ORIGIN**，说明这两块 section 内存地址是相连接的，则使用时可以缩减DRAM，增加hostif_buf_pool，或者缩减hostif_buf_pool，增加DRAM

收发通路buffer使用规则

1. **通路A** 数据包收发只使用 **buffer-level3**
2. **通路B** 消息包收发分两个方向，**主控->AIC-MCU** 方向均使用 **1624 长度**的 **buffer**，不同芯片此处有差异，

AIC8800MC：消息包方向:主控->AIC-MCU，所使用的是buffer-level3，与数据包通路A共享
AIC8800M40:消息包方向:主控->AIC-MCU，所使用的是buffer-level4，buffer-level4是独立静态内存，不包含在hostif_buf_pool section中

level	size	describe	number	describe
4	1624	不可调整	2	不可调整

3. **通路B** 消息包 AIC-MCU->主控方向，依据消息包的大小选择相应的buffer-level，**每个消息包都需要24字节的开销**，

```
e2a_msg_alloc(host_type, CUSTOM_MSG_CONNECT_IND, 0, 0, sizeof(struct custom_msg_connect_ind));
```

如上述接口 **e2a_msg_alloc** 中，**sizeof(struct custom_msg_connect_ind)** 为当前需要申请消息包的大小，接口 **e2a_msg_alloc** 实际需求的 **buffer-size** 为 **sizeof(struct custom_msg_connect_ind) + 24**，

```
a.buffer-size < buffer-level1.size时，使用buffer-level1
b.buffer-level1.size < buffer-size < buffer-level2.size时，使用buffer-level2
c.buffer-size > buffer-level2.size时，使用buffer-level3
d.如果需要申请buffer-level1，而buffer-level1数量已经使用完，则申请buffer-level2；需要申请buffer-level2时情况一致
```

4. **e2a_msg_alloc** 申请出来的 buffer 通过 USB/SDIO 总线发送，**AIC-MCU** 需要等待主控完成接收，才能进行释放再次使用
5. 按照默认配置，如果 **buffer-size > 128(buffer-level2.size)**，则发送该消息包需要使用 **buffer-level3**，则会与跑数据流竞争 buffer 资源，当数据流量大时，可能容易出现 "msg alloc err" 问题
6. 按照默认配置，如果 AIC-MCU 同时需要发送 6 个长度为 80 字节的消息包给主控，那么 **buffer-level2** 短时期内使用完，而 **buffer-level3** 可能正用于数据拉流无法申请到，那么就会报 "msg alloc err" 问题

收发通路buffer调整规则

- 通常情况下 **buffer-level1** 和 **buffer-level2** 的 **size** 和 **number** 不进行改动；
- 当出现 "msg alloc err" 时，可以对 **buffer-level1** 和 **buffer-level2** 进行调整，

```
a.size要求4字节对齐
b.buffer-level1.size < buffer-level2.size < 1624
```

```

tgt_cfg_wifi.h (aic8800-sdk\...\tgt_cfg) %
31:
32: /**
33:  * Hostif rx pkt filter mode
34:  * Current support:
35:  * 1) VNET_FILTER_DIRECT
36:  * 2) VNET_FILTER_SHARED
37:  */
38: #define CONFIG_HOSTIF_FILTER_MODE VNET_FILTER_DIRECT
39:
40: /**
41:  * Hostif tx msg1 buf number
42:  */
43: #define CONFIG_HOSTIF_RX_BUF_NUM_MSG1 4
44:
45: /**
46:  * Hostif tx msg2 buf number
47:  */
48: #define CONFIG_HOSTIF_RX_BUF_NUM_MSG2 2
49:
50: /**
51:  * Hostif tx msg1 buf size
52:  */
53: #define CONFIG_HOSTIF_RX_BUF_SIZE_MSG1 128
54:
55: /**
56:  * Hostif tx msg2 buf size
57:  */
58: #define CONFIG_HOSTIF_RX_BUF_SIZE_MSG2 256
59:
60: /**
61:  * Hostif total buf number
62:  */
63: #define CONFIG_HOSTIF_BUF_NUM_TOTAL 9
64:
65: /**
66:  * Hostif rx buf number threshold
67:  */
68: #define CONFIG_HOSTIF_RX_BUF_NUM_THRESHOLD 8
69:

```

调整方法是在编译 target 的 tgt_cfg_wifi.h 文件中添加 **buffer-level1.size**、**buffer-level1.num**、**buffer-level2.size**、**buffer-level2.num** 定义，否则使用 **hostif_cfg.h** 中默认的定义。修改这两个 **buffer-level** 时，客户可以自行依据业务消息包大小与数量进行调整

3. **buffer-level3.number** 的调整通常取决于客户实际的应用情况，

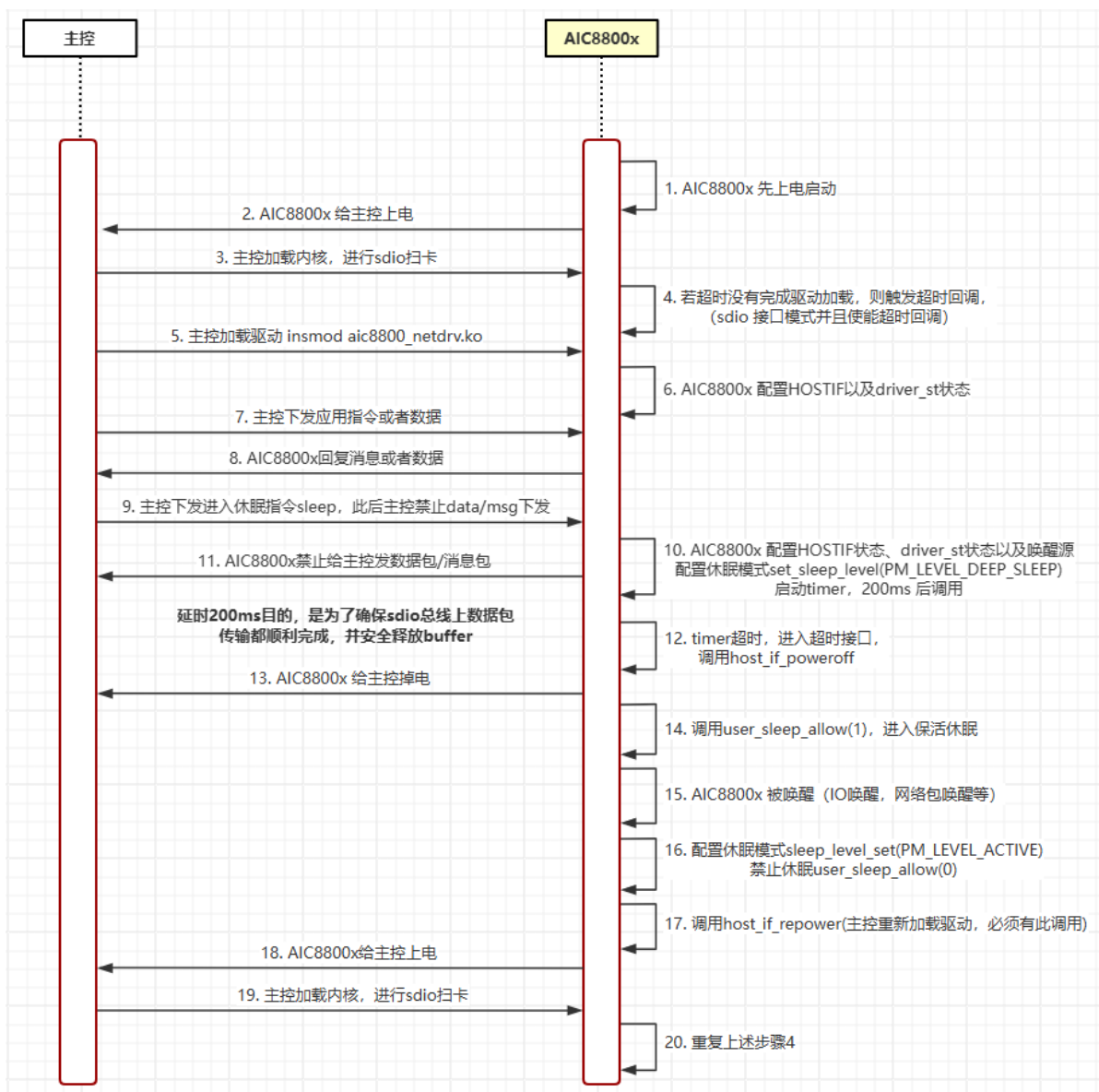
- a. DRAM 不够，则缩减 **hostif_buf_pool**，此时需要减小 **buffer-level3.number**
- b. DRAM 富余，则可以增加 **hostif_buf_pool**，加大 **buffer-level3.number**
- c. 修改 **CONFIG_HOSTIF_BUF_NUM_TOTAL** 时，**CONFIG_HOSTIF_RX_BUF_NUM_THRESHOLD** 需要做相应的增减

10. AIC-MCU低功耗-时序说明

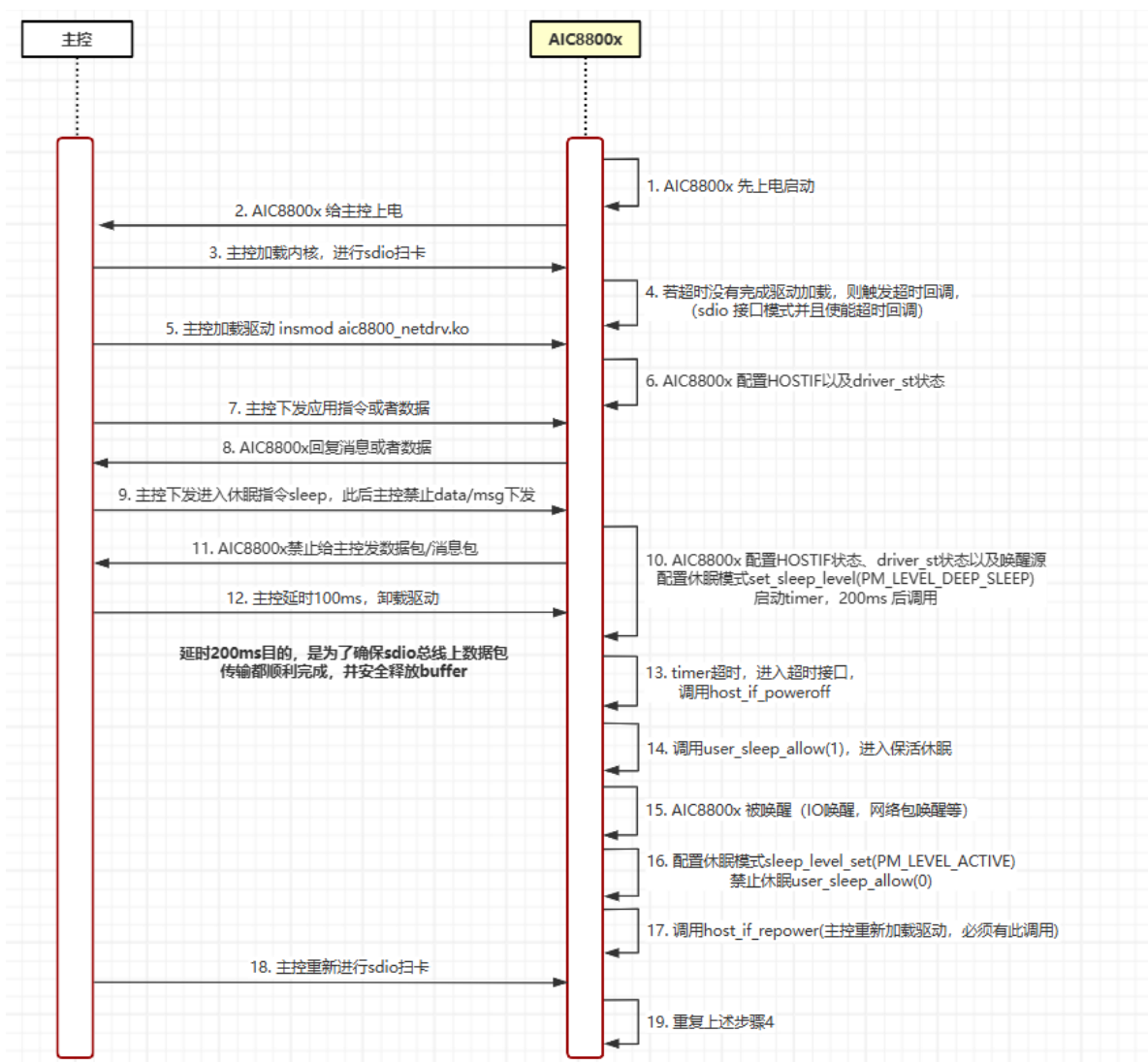
进入低功耗模式后，一般都是由 AIC8800x 做 WiFi 保活，或者 TCP 保活；而主控选择断电，等待退出低功耗模式，再重新上电，或者主控进入休眠模式，在退出低功耗模式时，重新唤醒。客户根据产品需求，选择相应的模式，本小节将对主控加载驱动、卸载驱动，与 AIC8800x 进入休眠、退出休眠需要调用的接口时序做进一步说明

以下时序说明以 sdio 接口为例，usb 相较于 sdio，不需要进行扫卡过程。

主控调电模式



主控不掉电模式



AIC-MCU的低功耗睡眠有不同的等级可供选择（常用等级）

芯片型号	休眠等级
AIC8800M40	PM_LEVEL_ACTIVE（不休眠）
	PM_LEVEL_LIGHT_SLEEP
	PM_LEVEL_DEEP_SLEEP
AIC8800MC	PM_LEVEL_ACTIVE（不休眠）
	PM_LEVEL_LIGHT_SLEEP
	PM_LEVEL_DEEP_SLEEP
AIC8800M	PM_LEVEL_ACTIVE（不休眠）
	PM_LEVEL_LIGHT_SLEEP
	PM_LEVEL_DEEP_SLEEP
	PM_LEVEL_HIBERNATE

支持唤醒IO请查看《AIC8800低功耗》文档

12. AIC-MCU低功耗-KeepAlive

在主控加载驱动之后，AIC-MCU 将与主控之间的接口状态设置为 **HOSTIF_ST_IDLE**，进一步依据 <5. AIC-MCU低功耗-状态说明> 更改接口状态。然而当主控 SDIO/USB 接口发生异常中断时，通常在这种情况下，AIC-MCU 无法自动识别，只能依靠看门狗进行复位。

为此，在主控与 AIC-MCU 之间增加 **Keep-Alive** 机制，主控定时发送指定的 msg 数据包，AIC-MCU 收到之后进行回复并更新心跳时间，同时 AIC-MCU 会启动定时器定时检查心跳时间是否及时进行更新，否则认为与主控断开了连接，之后将设置接口状态为 **HOSTIF_ST_INIT**

```
# fhostif_cmd.c

1. keep_alive 默认关闭
#define HOSTIF_KEEP_ALIVE 0

2. 心跳包时间设置（ms）
#define KEEP_ALIVE_PERIOD 250
```

13. AIC-MCU低功耗-主控DHCP

通常情况下，AIC-MCU 连接AP之后会自动进行DHCP获取IP，若需要由主控进行DHCP过程，则需要先给AIC-MCU配置虚拟的IP信息，等待主控获取到IP之后，主控再下发给AIC-MCU同步IP信息。


```
# fhostif_cmd.c
```

主控DHCP 默认关闭

```
#define HOSTIF_CNTRL_DHCP 0
```

14. AIC-MCU低功耗-OTA自动重启

通常情况下，AIC-MCU OTA升级之后不会自动重启，重启之后才会运行新的软件版本。

```
# fhostif_cmd.c
```

OTA自动重启 默认关闭

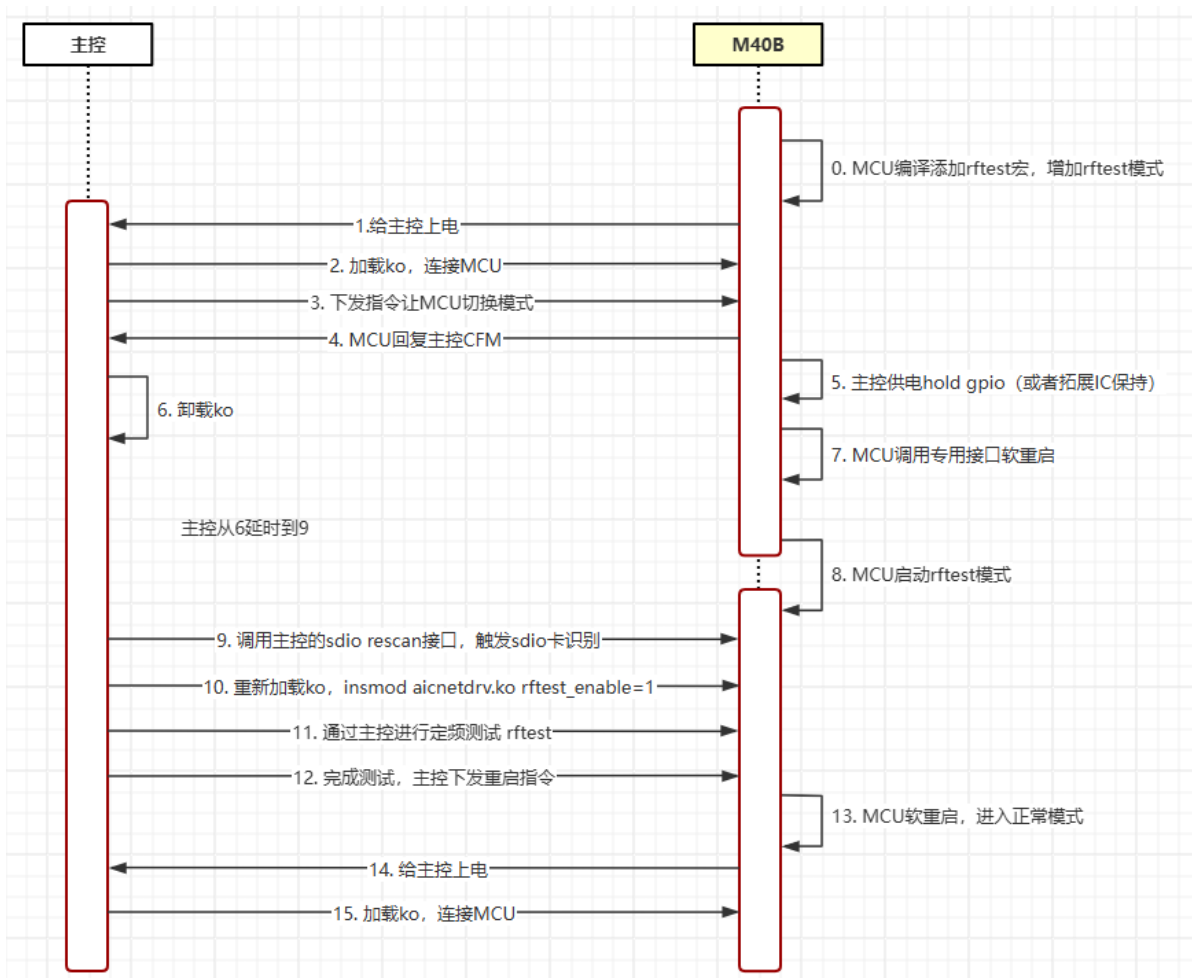
```
#define HOST_OTA_REBOOT 0
```

15. AIC-MCU低功耗-rftest切换

目前 AIC-MCU 进入定频有两种方式，方式1通过串口将 testmode.bin 下载到芯片内存中，之后由 AIC-MCU 芯片串口输入指令完成定频测试；方式2通过将 MCU 转化成 Device 模式，加载 linux 透传模式驱动，将 rftest 固件下载到 AIC-MCU 内存中，此时 AIC-MCU 与主控可以通过 SDIO/USB 进行交互，之后由主控应用层直接下发定频测试指令

而将 MCU 转化成 Device 模式，目前有两种方式，方式1是在 AIC-MCU 上电前由主控拉高B0，拉低B1；方式2是在 AIC-MCU flash 中下载 M2D.bin；在虚拟网卡模式应用，这两种方式均存在限制，主控IO必须连接上 AIC-MCU 的 B0/B1，给 AIC-MCU flash 下载 M2D.bin 也必须通过 AIC-MCU 的串口进行下载

针对主控和 AIC-MCU 只有 SDIO/USB 连接，以及定频测试指令需要在主控上下发的情景，虚拟网卡模式增加了 rftest 定频测试功能，具体实现流程如下



修改点

1. **aic8800-netdrv** 驱动编译时需要打开相应的宏 **CONFIG_RFTEST ?= y**
2. **aic8800-netdrv** 驱动加载时需要加上参数 **insmod aic8800_netdrv RFTEST_ENABLE=1**
3. **AIC-MCU target** 编译时需要打开相应的宏 **WIFI_RFTEST=on**

在虚拟网卡正常模式下, 主控通过下发 **CUSTOM_MSG_START_RFTEST_REQ**, 让 **AIC-MCU** 切换到 **rfctest** 模式, 切换接口实现详见 **custom_msg_start_rfctest_handler**

注意事项

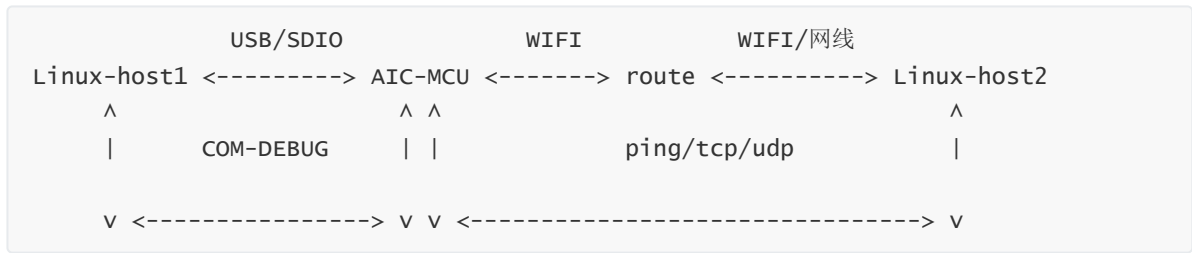
1. 系统正常关闭流程,

```
# a. Linux-host1 卸载驱动
sudo rmmod aic8800_netdrv
# b. Linux-host1 断电
# c. MCU 断电
```

2. 系统正常运行之后, 确保 **AIC-MCU** 供电稳定, 若中途 **AIC-MCU** 意外断电, 将导致 **SDIO/USB** 连接异常, **Linux-host1** 可能无法正常卸载驱动, 可能需要强制关机

二、rawdata模式

硬件设备连接说明



在此模式下，AIC-MCU相当于Linux-host1的USB/SDIO设备，可用于大量数据传输。

编译与启动

AIC-MCU

1. 更改tgt_cfg_wifi.h模式配置

```
/**
 * Hostif mode selection, match with host driver
 * Current support:
 * 1) HOST_VNET_MODE
 * 2) HOST_RAWDATA_MODE
 */
#define CONFIG_HOSTIF_MODE HOST_RAWDATA_MODE
```

2. 其余步骤同上述虚拟网卡

Linux主机

a. Linux驱动

1. 进入 /wifi/LinuxDriver/aic8800_netdrv 目录
2. 配置 Makefile 相关宏

```
# Driver mode support list
CONFIG_VNET_MODE ?= n
CONFIG_RAWDATA_MODE ?= y
```

3. 编译加载驱动

```
make -j8
sudo insmod aic8800_netdrv.ko
```

b. 应用程序

1. 进入 /wifi/LinuxDriver/app/nlaic_demo 目录
2. 编译运行

```
make -j8
# 运行应用程序必须添加权限
# demo-1
sudo ./nlaic_user 0 <char msg> <uint data> <bytes byte_ptr>
# demo-2
sudo ./nlaic_user 1
# demo-3
sudo ./nlaic_user 2 <uint block_size> <uint block_count>
```
